Victim Machine Setup Guide

Raspberry Pi Configuration for Recon & Password Cracking Labs

2026-10-30

Table of contents

1 Automated Setup (Recommended) 2

2 System Preparation 2
- 2.1 Update the Pi . . . 2
- 2.2 Force SHA-512 Password Hashing . . . 3
- 2.3 Assign a Static IP Address . . . 3

3 Setting Up User Accounts 5
- 3.1 Create labuser and Grant Sudo Access . . . 5
- 3.2 Create the Practice Accounts . . . 6
- 3.3 Plant the Trophy File . . . 6
- 3.4 Verify All Shadow Entries . . . 7
- 3.5 Verify labuser's Password is in rockyou . . . 8

4 Installing and Configuring SSH 8
- 4.1 Allow Password Authentication . . . 8

5 Installing and Configuring FTP 9
- 5.1 Install vsftpd . . . 9
- 5.2 Configure vsftpd . . . 9

6 Final Verification 10

7 Teardown After the Lab 11
- 7.1 Manual Teardown . . . 11

This document is for the **instructor only**. It covers everything needed to configure the Raspberry Pi as the victim machine before the lab sessions begin. Students should not see this document.

The Pi will expose two services – SSH and FTP – and host a target user account with a deliberately weak password. Students will discover the machine via nmap, attack it online with hydra, and later extract and crack its password hash offline.

> **Important**
>
> Complete all steps in this guide **before** students arrive. The two most time-sensitive steps are connecting the Pi to the LAN and noting its IP address, which students will need at the start of Lab 1.

1 Automated Setup (Recommended)

The companion script **`setup_victim.sh`** (in this folder) performs every step in this guide except assigning the static IP, which you still do by hand. It runs in two phases so that the deliberately weak configuration is **never exposed while the Pi is on the school/public network**.

Copy the script to the Pi:

```
scp setup_victim.sh <your-admin-user>@<pi-address>:~
```

Phase 1 – run now, while the Pi is still on the school network. Creates the four lab accounts with their login *disabled* (no passwords yet), forces SHA-512 hashing, and installs/configures SSH and FTP but leaves them locked down:

```
sudo ./setup_victim.sh
```

Phase 2 – run last, only after the Pi has been moved to the isolated, no-internet LAN. Sets the weak lab passwords, enables SSH password + root login, starts FTP, and prints the verification checklist:

```
sudo ./setup_victim.sh --go-live
```

`--go-live` refuses to run if the Pi still has a public IP address (override with `--go-live --force`). Set the static IP by hand just before this phase (see Section 2.3). Run `./setup_victim.sh --help` for the full description.

> **Note**
>
> The rest of this document is the manual, step-by-step equivalent of what the script does – use it to understand each step, to verify the result, or to set the Pi up entirely by hand. A companion `teardown_victim.sh` reverses it all when the labs are over (see Section 7).

2 System Preparation

2.1 Update the Pi

Start with a clean, up-to-date system:

```
sudo apt update && sudo apt upgrade -y
```

2.2 Force SHA-512 Password Hashing

The labs assume every password hash is **sha512crypt** (`$6$`, hashcat mode `1800`). Current Raspberry Pi OS does not produce `$6$` by default, and – this trips people up – *two different subsystems* choose the algorithm depending on how each password is set:

- **`chpasswd` and `useradd`** read `ENCRYPT_METHOD` from `/etc/login.defs` (default `YESCRYPT`, giving `$y$`).
- **interactive `passwd` and `adduser`'s password prompt** go through PAM (`/etc/pam.d/common-password`), which chooses the algorithm independently (often MD5, giving `$1$`, when no algorithm is named on the `pam_unix.so` line).

If you fix only one of them you get a *mix* of hash types. Do this **before creating any accounts** so every account below comes out as `$6$`:

```
# chpasswd / useradd path -- robust against a commented or differently-set line
sudo sed -i -E 's/^[# ]*ENCRYPT_METHOD\b.*/ENCRYPT_METHOD SHA512/'
 ↪  /etc/login.defs
grep -q '^ENCRYPT_METHOD SHA512' /etc/login.defs || echo 'ENCRYPT_METHOD SHA512'
 ↪  | sudo tee -a /etc/login.defs

# interactive passwd / adduser path -- PAM
sudo sed -i -E '/^password[[:space:]].*pam_unix\.so/
 ↪  s/\b(yescrypt|gost_yescrypt|md5|sha256|bigcrypt|blowfish|des)\b/sha512/g'
 ↪  /etc/pam.d/common-password
sudo sed -i -E '/^password[[:space:]].*pam_unix\.so/ { /sha512/!
 ↪  s/pam_unix\.so/pam_unix.so sha512/ }' /etc/pam.d/common-password
```

> **Tip**
>
> Regardless of the system default, you can always force the algorithm when setting a password with `chpasswd -c SHA512` (used throughout this guide). That is the surest way to guarantee a `$6$` hash on one specific account.

2.3 Assign a Static IP Address

The lab network is a flat `10.0.0.0/8`, and the Pi lives at the fixed address `10.7.7.200` within it. A static IP prevents the Pi from disappearing between lab sessions if the DHCP lease changes.

> **Why /8 here but /24 in the student scan**
>
> The Pi (and every lab machine) is configured with a `/8` mask so that, on a flat LAN, all `10.x.x.x` hosts are treated as directly reachable. Students, however, are told to *scan* only

`10.7.7.0/24` – every machine that matters sits in that `/24`, so the narrower scan finds them all without probing the other ~16 million addresses in the `/8`. Keep the interface mask (`/8`) and the scan scope (`/24`) distinct; they serve different purposes.

Current Raspberry Pi OS (Bookworm and later) manages networking with **NetworkManager**, not `dhcpcd`. The `/etc/dhcpcd.conf` file no longer exists on a fresh image, so configure the static address with `nmcli`.

First, find the name of the wired connection:

```
nmcli device status          # note the DEVICE (usually eth0)
nmcli connection show        # note the connection NAME (usually "Wired
 ↪  connection 1")
```

Set the static address, gateway, and DNS on that connection, then bring it back up to apply:

```
sudo nmcli connection modify "Wired connection 1" \
    ipv4.method manual \
    ipv4.addresses 10.7.7.200/8 \
    ipv4.gateway 10.7.7.7 \
    ipv4.dns 10.7.7.7
sudo nmcli connection up "Wired connection 1"
ip addr show eth0   # confirm the address is now 10.7.7.200
```

The setting is stored under `/etc/NetworkManager/system-connections/` and persists across reboots. If you prefer a menu-driven interface, `sudo nmtui` does the same thing interactively.

Older Pi OS images (Bullseye and earlier)

Images from before Bookworm (released October 2023) still use `dhcpcd`. On those, edit `/etc/dhcpcd.conf` instead and append:

```
interface eth0
static ip_address=10.7.7.200/8
static routers=10.7.7.7
static domain_name_servers=10.7.7.7
```

then `sudo systemctl restart dhcpcd`.

Note

`10.7.7.200` is used throughout both student lab documents as the Pi's address. If you use a different address, do a find-and-replace in both QMD files before printing.

3 Setting Up User Accounts

The Pi hosts four user accounts in total. **labuser** is the primary target for the online attack in Lab 1 and is also a sudoer, allowing students to escalate to root at the start of Lab 2. The remaining three accounts – **alice**, **bob**, and **carol** – exist only to populate the shadow file with hashes of varying difficulty for the offline cracking exercises.

The table below summarises all four accounts. Keep this page secure – do not share it with students.

User	Password	Cracking method	Notes
labuser	sunshine1	Hydra (Lab 1), then confirms setup in Lab 2	Must be in rockyou; used for SSH/FTP
alice	dragon	Dictionary (rockyou, no rules)	Near the top of rockyou – cracks in seconds
bob	c0bra	Rule-based (best64)	Leet form not in rockyou; best64's o->0 rule derives it from cobra
carol	abc123	Dictionary (rockyou, no rules)	Mid-list rockyou entry – takes a moment

3.1 Create labuser and Grant Sudo Access

```
sudo adduser labuser
# enter 'sunshine1' (or your chosen rockyou password) when prompted
```

Add labuser to the sudo group so students can escalate to root in Lab 2:

```
sudo usermod -aG sudo labuser
```

Verify the group membership:

```
groups labuser
# should include 'sudo' in the output
```

Test that escalation works:

```
su - labuser
sudo -i
# should drop into a root shell
exit
exit
```

> **Note**
>
> `labuser` having sudo rights is what allows the `sudo -i` command in the offline lab to succeed. Without this step, students will be denied when they try to read `/etc/shadow`.

3.2 Create the Practice Accounts

These accounts do not need sudo access – they exist only to add entries to the shadow file. They are created with a `nologin` shell so they cannot be used to log in, while their password hashes still appear in `/etc/shadow` for the offline cracking exercises.

```
# alice -- plain dictionary password
sudo adduser --disabled-login --shell /usr/sbin/nologin alice
echo "alice:dragon" | sudo chpasswd -c SHA512

# bob -- requires rule-based attack to crack
sudo adduser --disabled-login --shell /usr/sbin/nologin bob
echo "bob:c0bra" | sudo chpasswd -c SHA512

# carol -- plain dictionary password
sudo adduser --disabled-login --shell /usr/sbin/nologin carol
echo "carol:abc123" | sudo chpasswd -c SHA512
```

The `nologin` shell is what actually stops these accounts being used to log in – over SSH *or* FTP – so students cannot simply connect as alice, bob, or carol; they must crack the hashes offline. (On its own, `--disabled-login` only skips the initial password prompt; once `chpasswd` sets a password the account would otherwise be usable, which is why the shell is set explicitly.) Setting the shell does not affect the stored hash, so cracking still works exactly as described in the offline lab.

3.3 Plant the Trophy File

Lab 2 has students practise `find` and `grep` by locating a sensitive file in `labuser`'s home directory and downloading it with `scp`. Create that file now:

```
sudo mkdir -p /home/labuser/notes
sudo tee /home/labuser/notes/credentials.txt >/dev/null <<'EOF'
CONFIDENTIAL -- internal use only. Do not share.
```

```
Backup server login (reused from my main account, I know, I know):
  host     : 10.7.7.50
  username : admin
  password : hunter2-backup!

TODO: stop keeping passwords in plaintext files.
EOF
sudo chown -R labuser:labuser /home/labuser/notes
sudo chmod 600 /home/labuser/notes/credentials.txt
```

`labuser` owns the file, so students can retrieve it with the Lab 1 foothold alone – no root required. The contents are fictional; change them if you like, but keep a keyword such as `password` or `CONFIDENTIAL` so the recursive `grep` in the lab finds it, and keep the path at `/home/labuser/notes/credentials.txt` (it is named explicitly in the student lab).

3.4 Verify All Shadow Entries

```
sudo grep -E "labuser|alice|bob|carol" /etc/shadow
```

You should see four lines, each with a hash beginning with `$6$`. Confirm no entry has `*` or `!` in the hash field, which would indicate no password is set.

> **!** Important
>
> If any hash begins with `$y$` (yescrypt) or `$1$` (MD5) instead of `$6$`, the *Force SHA-512 Password Hashing* step above was skipped or did not take effect for that account – see that section for why the two subsystems disagree. The labs assume `$6$` (hashcat mode `1800`), so recreate all four passwords, forcing the algorithm explicitly:
>
> ```
> printf '%s\n' \
>   'labuser:sunshine1' \
>   'alice:dragon' \
>   'bob:c0bra' \
>   'carol:abc123' | sudo chpasswd -c SHA512
> ```
>
> Then re-run the `grep` above and confirm all four hashes now start with `$6$`. (Only as a last resort, if you genuinely cannot get `$6$`, you could instead change both student labs to the matching hashcat mode – `400` for `$y$`, `500` for `$1$` – but keeping everything on `$6$` is far simpler.)

3.5 Verify labuser's Password is in rockyou

```
grep -n "^sunshine1$" /usr/share/wordlists/rockyou.txt
```

This should return a line number. If it returns nothing, choose a different password from the suggestions below and update `labuser` accordingly:

```
sunshine1   dragon99   password1   princess   monkey123
```

Alice's and carol's passwords (`dragon` and `abc123`) are confirmed entries in rockyou and do not need checking. Bob's password (`c0bra`) is *intentionally* absent from rockyou in that exact form – its leetspeak spelling (`o->0`) does not appear in the list, which is what forces a rule-based attack. The base word `cobra` *is* in rockyou, and that is what lets `best64` derive `c0bra` from it.

4 Installing and Configuring SSH

SSH is almost certainly already installed and running on a Raspberry Pi OS image. Verify it:

```
sudo systemctl status ssh
```

If it is not running, install and enable it:

```
sudo apt install openssh-server -y
sudo systemctl enable ssh
sudo systemctl start ssh
```

Confirm it is listening:

```
sudo ss -tlnp | grep :22
```

> **Tip**
>
> `ss` (from the `iproute2` package) is the modern replacement for the deprecated `netstat`; it is preinstalled on current Pi OS. If you still prefer `netstat`, install it with `sudo apt install net-tools -y`.

4.1 Allow Password Authentication

Some Pi OS images disable password authentication by default in favour of key-based login. Hydra needs password authentication to be enabled.

```
sudo nano /etc/ssh/sshd_config
```

Ensure the following lines are present and uncommented:

```
PasswordAuthentication yes
PermitRootLogin yes
```

Setting `PermitRootLogin yes` is necessary for students to SSH in as root at the start of Lab 2 after cracking `labuser`'s password and escalating. Restart SSH to apply:

```
sudo systemctl restart ssh
```

> **!** Important
>
> `PermitRootLogin yes` is a significant security risk on any real system. This is acceptable here because the Pi is on an isolated LAN. Revert this setting when the lab is over.

5 Installing and Configuring FTP

Students already know vsftpd from the covert channels lab. Having a second service on the Pi makes the nmap output more interesting and gives hydra two protocols to attack.

5.1 Install vsftpd

```
sudo apt install vsftpd -y
```

5.2 Configure vsftpd

```
sudo nano /etc/vsftpd.conf
```

Find and set the following options:

```
anonymous_enable=NO
local_enable=YES
write_enable=NO
local_umask=022
chroot_local_user=YES
allow_writeable_chroot=YES
```

Setting `local_enable=YES` allows system users (including `labuser`) to log in with their system credentials. This is what hydra will attack.

Restart vsftpd:

```
sudo systemctl enable vsftpd
sudo systemctl restart vsftpd
```

Verify it is listening on port 21:

```
sudo ss -tlnp | grep :21
```

6 Final Verification

Before students arrive, run through this checklist:

1. Pi is connected to the LAN and reachable at `10.7.7.200`:

```
ping 10.7.7.200   # from another machine on the LAN
```

2. SSH is open on port 22 and accepts password login:

```
ssh labuser@10.7.7.200   # should prompt for password
```

3. FTP is open on port 21 and accepts `labuser` credentials:

```
ftp 10.7.7.200   # login as labuser
```

4. `labuser` can escalate to root:

```
ssh labuser@10.7.7.200
sudo -i           # should drop to a root shell without errors
exit
exit
```

5. All four shadow entries are present and hashed (not `*` or `!`):

```
sudo grep -E "labuser|alice|bob|carol" /etc/shadow
```

All four lines should have a `$6$` (or `$y$`) hash in the second field.

6. The `labuser` password appears in rockyou.txt:

```
grep -n "^sunshine1$" /usr/share/wordlists/rockyou.txt
```

This should return a line number. If it returns nothing, the password will not be crackable – choose a different one from the suggestions in the user accounts section above.

7. The practice account passwords are confirmed crackable:

```
grep -n "^dragon$"   /usr/share/wordlists/rockyou.txt   # alice
grep -n "^abc123$"   /usr/share/wordlists/rockyou.txt   # carol
```

For `bob`, the rule-based attack depends on two things being true: the leet form `c0bra` must be *absent* from rockyou, while the base word `cobra` must be *present* so `best64` can derive it:

```
grep -n "^c0bra$"   /usr/share/wordlists/rockyou.txt   # bob -- should return
 ↪  NOTHING
grep -n "^cobra$"   /usr/share/wordlists/rockyou.txt   # should return a line
 ↪  number
```

If `c0bra` ever turns up in your copy of rockyou, pick another leet word whose plain form is in the list but whose substituted form is not (e.g. `v1per` from `viper`) and update `bob` accordingly.

Once all seven checks pass, the Pi is ready.

7 Teardown After the Lab

When the lab sessions are over, revert the security-sensitive changes. The companion script **`teardown_victim.sh`** (in this folder) reverses everything `setup_victim.sh` did:

```
sudo ./teardown_victim.sh
```

It removes the lab SSH drop-in (disabling root/password login) and restarts SSH, stops and disables FTP (restoring the original `vsftpd.conf` if a backup was saved), and deletes the four lab accounts and their home directories – prompting once before the deletion. Use `--yes` to skip the prompt, or `--keep-users` to revert the exposure but keep the accounts. It deliberately leaves the static IP (revert that by hand) and the SHA-512 hashing settings (harmless) alone. Run `./teardown_victim.sh --help` for details.

7.1 Manual Teardown

To do it by hand instead:

```
# Disable root SSH login (remove the lab drop-in, or edit the main config)
sudo rm -f /etc/ssh/sshd_config.d/00-lab.conf
sudo systemctl restart ssh

# Remove all lab user accounts
sudo deluser --remove-home labuser
sudo deluser --remove-home alice
sudo deluser --remove-home bob
sudo deluser --remove-home carol

# Stop and disable vsftpd if not needed outside the lab
sudo systemctl disable vsftpd
sudo systemctl stop vsftpd
```

> **Note**
>
> If you set the Pi up entirely by hand (without `setup_victim.sh`), the SSH drop-in may not exist; in that case edit `/etc/ssh/sshd_config` directly and set `PermitRootLogin no` (and `PasswordAuthentication no` if you enabled it).